

Faculty of Computers and Information Systems

Information Retrieval
Practical Laboratory

Week 3: Text Preprocessing and Tokenization

Build the preprocessing function used throughout the retrieval labs. Inspect every transformation so that later index and query results can be explained.

Duration: approximately 2 practical hours. Suggested pacing: 15 minutes concept review, 25 demonstration, 50 guided implementation, 20 selected exercises, and 10 review. Finish remaining exercises after the session.

1. Lab Objectives

- Implement `tokenize(text)` using lowercase conversion and regex.
- Remove stop words using membership in a Python set.
- Construct a sorted collection vocabulary.
- Compare raw text, lowercase text, and token lists.

2. Required Background

Prerequisite: basic Python variables, conditions, loops, lists, dictionaries, sets, and functions. Complete the earlier labs before starting this week.

A document is one searchable unit. A token is an occurrence produced by splitting or matching text; repeated words create repeated tokens. In this course, a term is a normalized word retained for indexing. The vocabulary is the set of distinct retained terms across the collection.

The supplied approach is `words = re.findall(r"[a-z]+", text.lower())`. It selects consecutive ASCII letters and skips punctuation and digits. For example, Python, PYTHON becomes two python tokens, while C++ becomes c. It does not stem words, expand contractions, or support all languages.

A stop-word set lists words excluded by the course policy. The supplied set includes common words such as the and and, and also used and uses. Filtering a list preserves repeated content terms and their order. Do not turn tokens into a set before computing term frequencies in later weeks.

Documents and queries must use the same preprocessing. Otherwise a query term may fail to match the representation stored in the index. Keeping the preprocessing function unchanged across weeks makes comparisons interpretable.

Course material: The `tokenize` functions and stop-word set in `section-2.py` and `5.py`.

3. Python Concepts Used

- **str.lower:** Normalize uppercase letters to lowercase.
- **re.findall:** Extract letter sequences with `r"[a-z]+"`.
- **Set membership:** Use `word not in STOP_WORDS` to filter tokens.
- **List comprehension:** Build a new list while preserving order and repetition.
- **set.update and sorted:** Collect unique terms, then display them deterministically.

4. Worked Example

This complete small pipeline makes the token/term distinction visible. The full program uses the complete stop-word set from the supplied course files.

```
import re
stop_words = {"the", "and"}
```

```

text = "The Python, Python and DATA!"
words = re.findall(r"[a-z]+", text.lower())
tokens = [word for word in words if word not in stop_words]
print(words)
print(tokens)
print(sorted(set(tokens)))

```

The outputs are ['the', 'python', 'python', 'and', 'data'], then ['python', 'python', 'data'], and finally ['data', 'python']. There are three retained token occurrences but only two vocabulary terms.

5. Step-by-Step Practical Task

Step 1: Load the five documents in docs using the course loader.

Step 2: Print each original string and its lowercase form.

Step 3: Print the regex matches before filtering stop words.

Step 4: Implement tokenize(text), then print its returned list for each document.

Step 5: Accumulate all retained terms in a set and print the sorted vocabulary and its size.

Step 6: Test punctuation, repeated terms, digits, and text consisting entirely of stop words. Record the information lost at each stage.

Execution: open a terminal in this week folder and run `python lab03_solution.py`. On Windows, `py -3` can replace `python`. All required files are included; no internet or third-party Python packages are needed.

6. Complete Python Program

The listing below matches `lab03_solution.py` exactly. Keep indentation when copying. The exercise starter is separate and intentionally unfinished.

lab03_solution.py

```

import os
import re

STOP_WORDS = {
    "a", "an", "the", "is", "in", "for", "and", "are", "with",
    "like", "of", "to", "it", "be", "used", "uses", "that", "this",
    "was", "were", "by", "from", "on", "at", "or",
}

SAMPLE_DOCUMENTS = {
    "doc1": "Python is great for data science and machine learning",
    "doc2": "Java is used for enterprise applications and web development",
    "doc3": "Machine learning uses neural networks and decision trees",
    "doc4": "Python supports data analysis with libraries like Pandas",
    "doc5": "Java and C++ are used for game development and graphics",
}

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

def load_documents(folder_path):
    """Load UTF-8 .txt files; use five samples if missing or empty."""
    documents = {}
    if os.path.isdir(folder_path):
        for filename in sorted(os.listdir(folder_path)):
            filepath = os.path.join(folder_path, filename)
            if filename.endswith(".txt") and os.path.isfile(filepath):
                with open(filepath, "r", encoding="utf-8") as file:
                    documents[filename[:-4]] = file.read()
    if not documents:
        return SAMPLE_DOCUMENTS.copy()
    return documents

```

```
def tokenize(text):
    """Use the same ASCII-letter tokens as the supplied course files."""
    words = re.findall(r"[a-z]+", text.lower())
    return [word for word in words if word not in STOP_WORDS]

def main():
    documents = load_documents(os.path.join(BASE_DIR, "docs"))
    vocabulary = set()
    for name, text in documents.items():
        print("\nDocument:", name)
        print("Original:", text.strip())
        print("Lowercase:", text.lower().strip())
        print("Before:", re.findall(r"[a-z]+", text.lower()))
        tokens = tokenize(text)
        print("After:", tokens)
        vocabulary.update(tokens)
    print("\nVocabulary:", sorted(vocabulary))
    print("Vocabulary size:", len(vocabulary))

if __name__ == "__main__":
    main()
```

7. Expected Output

Representative output from the shipped collection and the default command. Calculated values come from a tested program run. Long output lines wrap here for printing; `expected_output.txt` contains the full terminal output. Selected rows are shown for large matrices.

```
Document: doc1
Original: Python is great for data science and machine learning
Lowercase: python is great for data science and machine learning
Before: ['python', 'is', 'great', 'for', 'data', 'science', 'and', 'machine',
'learning']
After: ['python', 'great', 'data', 'science', 'machine', 'learning']
...
Vocabulary size: 22
```

8. Laboratory Exercises

1. Basic: add a sentence containing repeated Python and data terms. Count tokens and unique terms separately.
2. Implementation: finish `tokenize` in the starter and replace its small practice stop-word set with the full course set.
3. Debugging: remove `lower()` temporarily and explain what the `[a-z]+` pattern does to Python and DATA.
4. Interpretation: compare tokenization of C++, e-mail, don't, and 2026. Explain the limitations without changing the required pipeline.
5. Implementation: compute the collection vocabulary from all document token lists.
6. Extension: compare vocabulary sizes with and without stop-word removal. Report specific terms responsible for the difference.

9. Student Tasks

- Your completed `tokenize` function and a before/after trace for two documents.
- The sorted vocabulary and counts of retained token occurrences and unique terms.
- Three edge-case tests with expected and actual token lists.

Submit your own implementation and evidence. Do not submit an untouched starter as a completed solution. Label all outputs with the query or input that produced them.

10. Review Questions

1. How does a token occurrence differ from a vocabulary term?
2. Why should the same tokenizer process documents and queries?
3. Why is a set suitable for stop words?
4. What information is lost when tokens are converted to a set?
5. What happens to digits and punctuation under `[a-z]+`?

11. Challenge Exercise

Optional: Add a separate optional tokenizer for Unicode words and compare it with the required ASCII tokenizer on accented and Arabic text. Keep the original implementation as the baseline and explain the effect on vocabulary.